schedsim

Melih Efe Sönmez
İlhan Altınay

1 Project Overview

- **What our scheduler does:** This project, which is a GNU assembly program, implements five different CPU scheduling algorithms. The program takes an input line and produces an output line that shows clock-by-clock execution trace of the input line depending on the head of the line which determines the algorithm used.
- **Supported algorithms (FCFS, SJF, SRTF, PF, RR):**
  - **FCFS (First Come First Serve):** Processes are executed by earliest arrival time, non-preemptively.
  - **SJF (Shortest Job First):** Processes are executed by shortest burst time, non-preemptively. (Processes arrives at $t = 0$)
  - **SRTF (Shortest Remaining Time First):** Processes are executed by shortest remaining time, preemptively.
  - **PF (Priority First):** Processes are executed by lowest priority number, preemptively.
  - **RR (Round Robin):** Processes are executed by cyclic order, preemptively. It processes quantum times head of the queue and adds the remaining part of the process to end of the queue (Processes arrives at $t = 0$)
- **Overall execution flow from input to output:** The general execution flow of the program is as follows: The _start label is the entry point. First, the input is read into a 256-byte buffer named input_buf using the read syscall. Then, it determines which algorithm to call by examining the first (if necessary the second) character of the input. The selected algorithm function parses the input according to its own format (parse_input), stores the process data in a 16-byte memory named proc_buf, and sorts them using sort_processes. Afterwards, main loop of the algorithm executes. Then, ID of the selected process or the character X is added to output_buf via append_char. When all processes are completed, the program calls the print_output function which writes the output to stdout with a single write syscall, and the program terminates with the exit syscall.

2 Parsing Logic

2.1 Input Processing Strategy

- **How tokens are separated (spaces, hyphens):**

  The tokens are obtained in two steps. Firstly, space character are detected to separate tokens which are algorithm name, process descriptors. Secondly, hyphen characters are detected to separate process descriptors. For example, **FCFS A-3-1 B-2-2** is separated into three pieces by spaces firstly, and A-3-1, B-2-2 is separated into three pieces by hyphens secondly.

  The parse_input function operates in the first step. By the .pi_skip_spaces, the input pointer (%rsi) goes forward by skipping spaces. When it reaches the next meaningful character, it checks whether the input has ended by the .pi_check_end. The field parsing within the token boundary is delivered to the parse_one_process function which reads the ID, then parses a numeric field after each hyphen.

- **How algorithm type is detected:**

  When the input is read after the _start label, the first byte of input_buf is loaded into the %al register, and first letter is detected. When S is detected (due to the fact that SJF and SRTF start with the same letter), the second byte is checked (movb 1(%rdi), %al) If it is J, it is SJF; if it is R, it is SRTF. A jump to the corresponding algorithm is executed by je (jump if equal) instruction, and the algorithm function is called.

  After the jump, each algorithm calls parse_input call. Before the call, it goes forward %rsi by the length of the algorithm name (addq $4, %rsi for SJF, addq $5, %rsi for FCFS/SRTF). Thus, the parser does not have to process the algorithm name again.

- **How numeric values are parsed from character:**.

  They are parsed by a helper function named atoi_advance. This function works by "shift-and-add" approach: an accumulator is initialized to zero, then in each loop the character is read, its numeric value is obtained by subtracting 0, the accumulator is multiplied by 10, and the new digit is added. When the character is not in the 0-9 range, the loop terminates and the result is delivered to %rax. The function goes forward the %rsi pointer externally (inc %rsi), which means when the function returns from atoi_advance, it finds the pointer exactly on the first non-digit character.

2.2 Register-Level Parsing Design

- **Which registers hold pointers (e.g., input buffer):**

  %rsi — input buffer pointer. It is the only pointer that goes forward on input_buf throughout the parsing process. parse_input, parse_one_process, and atoi_advance share the same %rsi; each function uses it as much as it reads/skips.

%r12 — (in parse_one_process) target process pointer. It holds the starting address of the 16-byte process currently being processed in proc_buf. The fields are written with the offsets (%r12), 4(%r12), 8(%r12), and 12(%r12).

%r12 — (in parse_input) the next process pointer. After each process is parsed, it is shifted to the next process using addq $16, %r12. The use of the same register for different purposes in the two functions is protected with push/pop.

- **Which registers hold temporary numeric values:**

  %rax — the accumulator and return value of atoi_advance. The number accumulates here, and the result is read from here when the function ends.

  %ecx — the numeric value of a single character in atoi_advance. The character is zero-extended loaded with movzbl (%rsi), %ecx, and then converted to the 0-9 range with subl $'0', %ecx.

  %r13 — the field count in parse_input. Depending on the algorithm, it takes the value 2 (SJF, RR), 3 (FCFS, SRTF), or 4 (PF), and is delivered as a parameter to parse_one_process. The same parser is reused for different algorithms by this count.

  %r14 — the quantum accumulator. When it is detected that a token has no hyphens, the value returned from atoi_advance is copied into %r14. When parsing is finished, it is moved to %rax and becomes the return value of the function. RR algorithm uses this value.

  %rbx — the token determiner pointer. To determine whether a token is a process definition or a quantum, a forward scan is performed with %rbx without changing %rsi. If a hyphen is found, it is a process, and if a space/end is found without finding a hyphen, it is a quantum.

- **How digits are accumulated into integers:**

  The core three instructions of atoi_advance are as follows:

```
imulq   $10, %rax   # multiply the current number by 10
addq    %rcx, %rax  # add the new digit
inc     %rsi        # advance the pointer by one character
```

  That is, for the input "123", the steps $0 \to 1 \to 12 \to 123$ occur respectively. Loop control is executed with an unsigned comparison after subtracting the character from '0' (cmpl $9, %ecx followed by ja .atoi_done). In this way, any non-digit character is detected with a single check. (After subl $'0', %ecx, the '-' character has the value $45 - 48 = -3$; but as an unsigned value, it is a very large number and is caught by ja.)

- **How delimiters (' ', '-') are handled:**

  Space (' '): It is the token delimiter. The .pi_skip_spaces loop skips spaces until it reaches the next meaningful character. As this loop runs before each token, the parser does not break when there are multiple spaces.

  Hyphen (-): It is a field separator within the token. It is skipped with a single inc %rsi before each numeric field in the parse_one_process function, since atoi_advance

already leaves the pointer on the hyphen. After the ID field, two inc %rsi instructions are executed: one to pass the ID character, and one to pass the hyphen.

How our parsing avoids corruption and maintains correctness:

- **Preservation of callee-saved registers:** At the beginning of the parse_input function, %rbx, %r12, %r13, and %r14 are pushed, and they are popped at the end. parse_one_process protects the %r12 and %r13 it uses. Thus, the calling algorithm function (e.g., srtf_algo) gets its own %r12–%r15 back correct after parsing.
- **Zeroing of the slot:** When parse_one_process starts executing, the 16 bytes of the target process are zeroed using movq $0, (%r12) and movq $0, 8(%r12). This provides that fields not read by the algorithm (e.g., arrival and priority for SJF) are guaranteed to be 0.
- **A single read head:** Since all functions share %rsi, there cannot be position inconsistency. Wherever one function leaves %rsi, the next function continues from there.

3 State Representation (Memory Layout)

The entire state of the program is maintained in fixed-size memory blocks defined in the .bss section within schedsim.s. Because there is a limit of a maximum of 10 processes and 1024 output characters, all buffers are allocated at compile time, there is no dynamic memory allocation.

3.1 Process Representation

Each process is stored in memory as a 16-byte block, and proc_buf (.space 160) is the sequential array of these blocks. This structure was preferred over parallel arrays because when multiple operations are performed on a process (remaining time check, decrement, ID reading), accessing via offsets from a single base register produces the fewest instructions. The transition from process address to index is done in a single instruction with shl $4, %eax.

Internal layout of the 16-byte slot:

Offset	Size	Field
0	1 byte	ID character ('A', 'B', ...)
1–3	3 bytes	(alignment padding)
4	4 bytes	Burst / remaining time
8	4 bytes	Arrival time
12	4 bytes	Priority

The ID is 1 byte, but it is padded by 3 bytes so that the next integer field starts at its natural 4-byte alignment. Burst and remaining time share the same field, and parsing

writes the burst. SRTF/PF/RR decrement this same field with decl 4(%rsi) or subl since FCFS and SJF are non-preemptive, they only read this field. parse_one_process zeroes out the 16 bytes before writing to the slot; thus, unused fields are 0.

3.2 Global State

- **current time:** Each algorithm keeps its own local current time in the %r13d register. It is stored in the register because writing to memory and reading from memory in every cycle of a loop creates unnecessary memory operations.
- **ready queue / process list:** There is no separate ready queue for FCFS, SJF, SRTF, and PF. All processes are within proc_buf, and the "readiness" state (arrival $\leq$ current_time and remaining > 0) is checked during the scan in every cycle. On the other hand, there is an explicit queue for RR: rr_queue: .space 64 (16×4 bytes) operates as an integer array which runs as a circular buffer. The queue holds indexes pointing to proc_buf. rr_head and rr_tail are counters, and the actual index is obtained using andl $15, %ecx.
- **output buffer:** Allocated with output_buf: .space 1024). The output is accumulated character by character: append_char writes each character to output_buf[output_pos] and increments output_pos by 1. In addition, '\n' is also added as the final character in print_output. A single write syscall is performed throughout the entire execution.
- **counters (remaining processes):** process_count (4 bytes) is kept in memory because it is shared between the parser and the algorithms. Counters in algorithms (e.g., %r12d = time worked actually, %r13d = current_time) stored in registers. In short, values that are read and written repeatedly throughout the function are kept in registers, while values that are shared across functions are kept in memory.

3.3 Mandatory Diagram (Hand-Drawn)

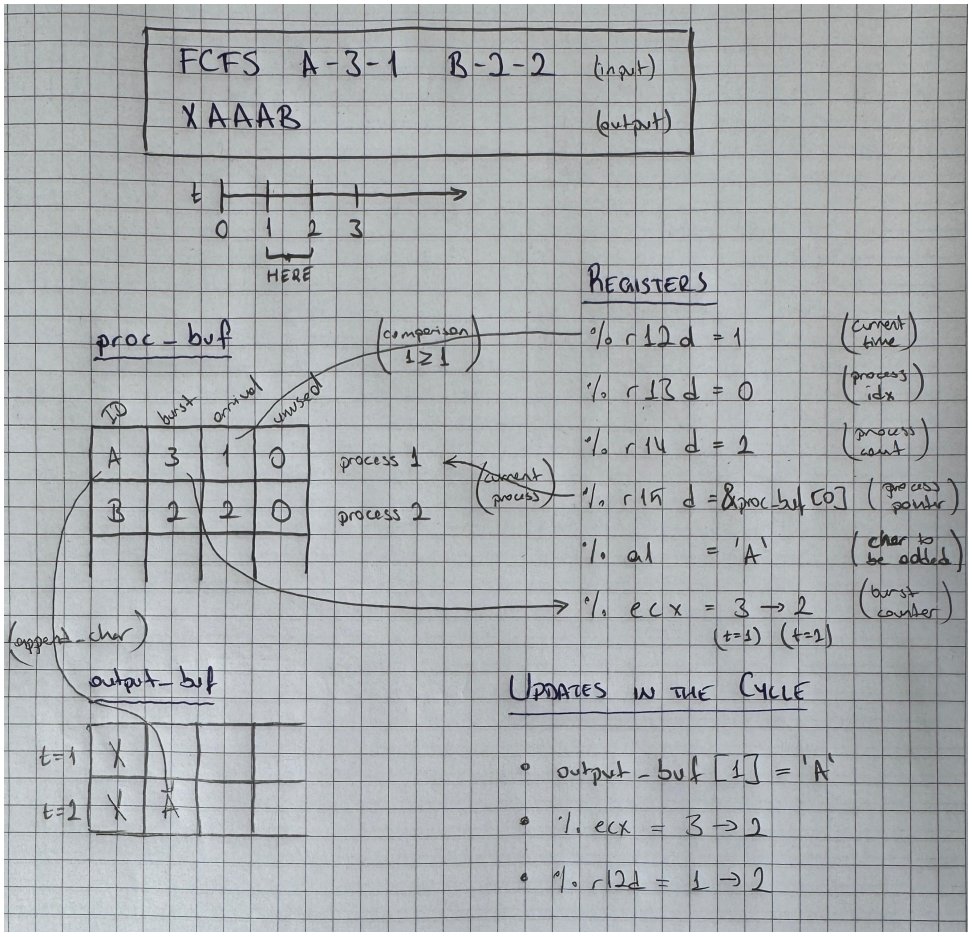


Figure 1: One Clock Cycle Diagram

4 Register Usage Strategy

- **Registers are used for**
  - **loop counters:**
    - **%ecx** — inner scan loop counter. In SRTF, PF, and RR, it is incremented from 0 up to process_count in the scan_loop which is performed on the process list during every cycle.
    - **%r13d** — outer loop counter. It operates as the process index in FCFS and SJF, and as current_time in SRTF and PF. These two roles are given to the same register because both represents the progress counter of the main loop.
    - **%ebx** — sorting inner counter (index j within sort_processes)
  - **pointers:**
    - **%rsi** — primary read pointer. It traverses input_buf during parsing. It holds the process address of the being handled within proc_buf, in the algorithm functions. (i.e. the address of data read)

    - **%rdi** — write pointer / function parameter. It is the write pointer to output_buf within print_output and append_char. Also, it is the target process pointer in the parse_one_process call.
    - **%r12** — process pointer (write destination in the parser)
    - **%r15** — process pointer (in FCFS, SJF) and quantum (in RR)
  - **current process index:**
    - **%r11d** — index of the selected process in SRTF and PF. It is initialized with -1 at the start of the scan. If it is still at the end of the scan, the CPU is idle.
    - **%r14d** — process index dequeued from the queue in RR.
  - **temporary computations:**
    - **%rax** — digit accumulator (atoi_advance), parameter to append_char (%al), intermediate value in address calculations.
    - **%edx, %edi** — instantaneous comparison operands (remaining, arrival, priority values).
    - **%r10d** — best so far. The smallest remaining in SRTF, the smallest priority in PF. It is initialized with 0x7FFFFFFF, so the first real candidate always beats it.
    - **%r9d** — best so far remaining in PF for tie-break.
    - **%r8–%r11** — four 8-byte temporaries while swapping 16-byte structs within sort_processes.

- **Registers which are preserved across function calls:** callee-saved registers are %rbx, %rbp, %r12–%r15; caller-saved registers are %rax, %rcx, %rdx, %rsi, %rdi, %r8–%r11. All five algorithm functions push %rbx, %r12–%r15 at the beginning and pop them at the end. parse_input and sort_processes protect the callee-saved registers they use. parse_one_process only pushes %r12 and %r13 which it uses.

  In append_char, %rcx and %rdi are pushed although they are caller-saved. This design's goal is to simplify the calling code. %rcx and %rdi generally hold temporary values within the algorithm loops. If append_char changed them, the calling code would have to do a manual push/pop before/after every call.

- **How we avoid accidental overwrites:**
  - **Role-based consistency:** the same register is allocated for the same mission in every file.
  - **callee-saved register usage:** callee-saved registers are protected at the beginning/end of the function, so the values in the calling function are safely stored. caller-saved registers are not used for persistent data. Values which exists in a loop are kept in %r12–%r15, otherwise there was a risk of losing them after every append_char call.

callee-saved / caller-saved: Registers are divided into two classes: callee-saved (%r12–%r15, %rbx, %rbp) and caller-saved (%rsi, %rdi, %r8–%r11, %rax, %rcx, %rdx). Callee-saved registers are registers that must be preserved by the function itself. If the function

is going to modify them, it must push them at the beginning and pop them at the end. So, the calling code knows that these registers have the same value before and after the call. Caller-saved registers might be modified after a function call, and the caller who wants to preserve them must push them. In our implementation, our five algorithms push %rbx, %r12–%r15 at the beginning and pop them at the end. parse_input and sort_processes protect the callee-saved registers they use.

5 Scheduling Logic

5.1 Clock Cycle Simulation

Our program simulates CPU scheduling clock cycle by clock cycle. In each time step, the following steps happen:

- **Checking Arrivals:** First, we compare the `current_time` (`%r13d`) with the `arrival_time` of each process in our `proc_buf`. If a process has arrived (`arrival_time <= current_time`) and still has work to do (`remaining_time > 0`), it is ready to run.
- **Picking a Process:** The way we select the next process depends on the algorithm. For FCFS and SJF, we just take the next process in our sorted array. For SRTF and PF, we use a loop to check all ready processes and find the best one (like the one with the lowest remaining time).
- **Updating Variables:** After picking a process, we decrease its `remaining_time` using `decl 4(%rsi)`. Then, we increase both the `current_time` and our total work counter (`%r12d`).
- **Writing the Output:** We put the 1-byte ID character of the chosen process into the `%al` register. If no process is ready (CPU is idle), we put `'X'` in `%al` instead. Finally, we call the `append_char` function to save this character into our `output_buf`.

5.2 Algorithm Implementations

- **FCFS (First Come First Serve):** Before the simulation starts, we sort the `proc_buf` by `arrival_time` using Bubble Sort. Then, we simply go through the array and run each process until its burst time becomes zero. We used an outer loop for the processes and an inner loop for the clock cycles.
- **SJF (Shortest Job First):** Since all processes arrive at time 0 in SJF, we sort them by `burst_time` instead of arrival time. After sorting, it works exactly like FCFS. It runs each process fully without stopping.
- **SRTF (Shortest Remaining Time First):** This algorithm is preemptive. At every single clock cycle, a loop checks all available processes. We keep the smallest remaining time in `%r10d` and the best process index in `%r11d`. If a new process arrives with a shorter time, we just update `%r11d` to switch to it.

- **PF (Priority First):** This is similar to SRTF, but we check the priority first using the `jl` (jump less) instruction. If priorities are the same, we check the remaining times. If those are also the same, our loop naturally picks the one that came first in the input because we scan from left to right.
- **RR (Round Robin):** We used a circular queue (`rr_queue`) for this algorithm. We keep the quantum value in `%r15d`. We take a process from the head of the queue and run it. If it still has work after its quantum finishes, we push it back to the tail. If it finishes early, we calculate the remaining quantum and add `'X'` characters to the output using a small padding loop (`.rr_pad`).

6 Control Flow Design

The control flow of the program is structured in a modular way, adhering to system programming principles. The program lifecycle consists of three main phases: *Parsing*, *Scheduling*, and *Output*.

- **Parsing Phase:** Starting at the `_start` label, the program first reads the input into `input_buf` using the `sys_read`. The main dispatcher compares the first few characters of the algorithm name (`cmp`, `je`) to route the execution to the corresponding algorithm subroutine.
- **Scheduling Phase (Simulation Loop):** Each algorithm contains its own main execution loop. For SRTF, PF, this loop simulates every single clock cycle (`current_time`), whereas for FCFS, SJF, it iterates over processes chronologically. The control flow heavily relies on `test` and `cmp` to check process states, followed by conditional jumps like `jz` or `jnz` to decide whether to append an 'X' or execute a process.
- **Loop Patterns and Jumps:** Loop counters are initialized using `xor %ecx, %ecx`. Loop boundaries are managed with a combination of `incl %ecx`, `cmpl %edx, %ecx`, and `jl`. For complex decision-making, such as PF tie-breaking, nested `jl`, `jg`, and `jge` instructions are used.
- **Termination Condition:** The simulation loop terminates either when the total active CPU cycles match the sum of all burst times or when the process queue becomes completely empty in RR.
- **Output Phase:** During the simulation, `append_char` is repeatedly called to write data to `output_buf`. Once the simulation concludes, the program transitions to the `print_output`, which issues a single `sys_write` call to flush the buffer to stdout, followed by `sys_exit` to terminate.

7 Representative Code Snippet

The following snippet from `pf.s` demonstrates the core preemptive selection and multi-stage tie-breaking logic.

Listing 1: Priority First Tie-Breaking Logic

```
    # Compare priority first (lower value = higher priority)
    movl    12(%rsi), %edi  # Load priority of current process
    cmpl    %r10d, %edi     # Compare with best priority found
    jl      .pf_replace     # Strictly better priority -> update best
    jg      .pf_skip        # Strictly worse -> ignore

    # Priorities equal -> compare remaining time (SRTF behavior)
    movl    4(%rsi), %edx   # Load remaining time
    cmpl    %r9d, %edx      # Compare with best remaining time
    jge     .pf_skip        # remaining >= best, keep earlier index

.pf_replace:
    movl    %edi, %r10d     # Update new best priority
    movl    %edx, %r9d      # Update new best remaining time
    movl    %ecx, %r11d     # Store index of the best process
```

Explanation: This snippet shows the core decision-making of the scheduler. The `%rsi` register points to the current process data in memory, while `%r10d` holds the lowest priority found in the scan so far. First, the priority is compared. If a tie occurs, the logic falls through to the SRTF rule, comparing the remaining times via `%r9d`. The use of the `jge` jump instruction ensures that if both priority and remaining times are equal, the original process is kept, handling the stable selection requirement.

8 Testing and Debugging

8.1 Testing Strategy

Our testing strategy combined manual and automated validation. Initially, manual testing was performed by feeding standard inputs via terminal (e.g., ./schedsim ¡ input_01.txt). Once the output format was stabilized, we transitioned to the automated testing provided in the `Makefile`. We used the `make testcases` command to execute the program against a bunch of inputs, routing the standard output into a dedicated `my-outputs` directory. Finally, `make grade` was used to compare our outputs against the expected outputs.

8.2 Debugging Approach

Our debugging process relied on low-level techniques:

- **Register Inspection:** We used GDB (GNU Debugger) to set breakpoints at critical loops (e.g., `.pf_scan_loop` and `.rr_turn_loop`) to inspect register values. We monitored registers like `%r10d` (best priority) and `%ecx` (loop counters) to verify that state updates were correct.

8.3 Edge Cases

Our implementation is designed to handle several edge cases defined in the project:

- **Multiple Arrivals at the Same Time:** Handled universally across non-preemptive algorithms using a bubble sort logic. The `jle` condition in `sort_processes` ensures that processes with identical key fields do not swap, thus preserving their original input order.
- **Identical Priorities (PF):** Handled via a multi-stage comparison chain. If priority values are equal, the code falls back to comparing remaining burst times (SRTF behavior). If those are also equal, the scan loop prefers the process encountered earlier in the array, maintaining deterministic behavior.
- **Quantum Partially Used (RR):** If a process finishes before its quantum expires, the remaining CPU cycles in that quantum slot must not be given to the next process. Our `.rr_pad` label calculates the difference and runs a loop to append the exact number of 'X' characters to the output buffer before popping the next process from the queue.
- **CPU Idle Periods:** Handled explicitly in preemptive algorithms by checking if any process is eligible to run. For instance, in SRTF and PF, if the scanner finds no process that has arrived and has remaining work (indicated by `%r11d == -1`), the execution jumps to the `idle` label, appends an `'X'` to `output_buf`, increments the global `current_time`, and continues to the next cycle.

9 Course Concepts

Our implementation relies on fundamental system programming concepts discussed throughout the course, translating high-level abstractions into assembly.

9.1 Memory and Data Layout

Instead of using dynamic memory like `malloc`, we reserved fixed spaces in the `.bss` section. We store processes one after another in `proc_buf`. Each process acts like a struct and takes exactly 16 bytes. Inside these 16 bytes, fields like ID, burst time, and arrival time take 4 bytes each. This layout makes it easy to find data using simple math, like using `movl 4(%rsi), %ecx` to quickly read the burst time.

9.2 Registers and Low-Level Computation

We followed the standard x86-64 rules and used callee-saved registers (like `%rbx` and `%r12`-`%r15`) so our values wouldn't get erased when calling other functions. To make the program run faster, we kept important variables (like `current_time` in `%r13d`) inside registers instead of reading them from memory every time. We also used basic ALU instructions like `testl %ecx, %ecx` to quickly check values before jumping.

9.3 Control Flow and Branching

Since assembly doesn't have for loops or if else statements, we built them ourselves using compare (`cmp`) and jump instructions. For loops, we used `%ecx` as a counter and checked it against a limit. For complex decisions, like the 3 step tie-breaker in PF, we chained jump instructions like `jl` and `jge`. This helped us check multiple conditions without needing high-level AND/OR operators.

9.4 Parsing Without Libraries

Because we couldn't use the C library, we had to convert strings to integers ourselves. We wrote a function called `atoi_advance` that reads bytes one by one. To get the actual number, we subtract the ASCII value of `'0'`. Then, we multiply our total by 10 (`imulq $10, %rax`) and add the new digit. We also checked for spaces and hyphens manually (like `cmp $'-', %al`) to carefully skip over them.

9.5 Tradeoffs

- **Simplicity vs. Efficiency:** We decided to use Bubble Sort instead of a faster algorithm like QuickSort. Since the project limits us to 10 processes, the speed difference doesn't matter. More importantly, Bubble Sort is inherently "stable" (thanks to `jle` check), which automatically handles the rule of keeping processes in their original input order when there is a tie.
- **Readability vs. Performance:** We grouped each process's data together into 16-byte blocks instead of creating separate parallel arrays for IDs, burst times, etc. Separate arrays might be slightly faster for the CPU cache in some specific loops, but grouping them made our code much easier to read and made swapping processes during the sort much simpler.
- **Static vs. Dynamic Design:** We hardcoded the sizes for our memory buffers (like 1024 bytes for the output buffer and 64 bytes for the RR queue) instead of asking the OS for memory while the program runs. Since the assignment constraints are fixed, this static approach makes our program faster and prevents memory overflow bugs.

10 Conclusion

In conclusion, writing this CPU scheduling simulator entirely in x86-64 assembly taught us a lot about low level state management and how computers handle memory.

- **Strongest Design Decision:** The best choice we made was grouping each process's data into 16-byte blocks in the `.bss` section and pairing it with a simple Bubble Sort. Since Bubble Sort is stable, it naturally handled the tricky tie-breaking rules without us needing to write extra code to track it. Also, using a bitwise AND

operator (`andl $15, %ecx`) to wrap around our Round Robin circular queue was a trick that completely prevented any buffer overflow bugs.

- **Biggest Challenge:** Not being allowed to use the standard C library made the very beginning of the project the hardest part. Writing our own `atoi_advance` function to read numbers with multiple digits, while carefully moving the pointer (`%rsi`) past spaces and hyphens, was tricky. It required us to keep very strict track of our registers and spend a lot of time debugging step by step with GDB.
- **Possible Improvements:** Using Bubble Sort and scanning through arrays ($O(N^2)$) works perfectly fine since we only ever have up to 10 processes. However, for a real operating system, this would be too slow. A good future improvement would be using a MinHeap for the SRTF and PF algorithms. This would drop the time it takes to find the best process from $O(N)$ down to $O(\log N)$.

AI Usage Statement

We confirm that the assembly code, structural design, and technical logic presented in this project were entirely developed by us.

- **Tools Used:** Gemini 3 Flash, Gemini 3 Pro, Claude Sonnet 4.6
- **Purpose:** AI assistance was utilized strictly for grammar correction, academic phrasing, and structuring the LaTeX formatting of this documentation. Also, used for code formatting for coding section.
- **Accountability:** We fully understand, can explain, and take complete responsibility for every line of assembly code and every technical concept detailed in this report.